

NHS: Analysing Staffing and Resource Utilization in the Healthcare Network

Overview

The United Kingdom has undergone a profound transformation due to the Covid-19 pandemic, impacting various aspects of life, including education, the economy, and the healthcare system. This study specifically delves into the challenges faced by the National Health Service (NHS) between January 2020 and June 2022 in managing the escalating number of Covid-19 cases. Notably, the study places significant emphasis on the obstacles faced, particularly in the aftermath of the easing of Covid-19 restrictions starting from August 2021.

The central focus revolves around aligning infrastructure and resources with the rising demand. The analysis is concentrated on exploring service settings, appointment modes and types, scrutinizing healthcare professionals and appointment trends, assessing resource and capacity utilization, and tackling the issue of missed appointments.

Business Objectives

Efficiently allocate resources, address capacity needs, and reduce missed GP appointments to improve the NHS' effectiveness in England.

Analytical Approach

The analytical methodology commenced with rigorous data cleaning procedures to ensure the integrity of the 'ad,' 'nc,' and 'ar' datasets. This involved meticulous checks for missing values, the removal of duplicates (21,604 duplicates in 'ar'), and the incorporation of external data ('icb_names') to enrich the dataset with valuable regional information. Importing essential libraries for the project was a foundational step, with Pandas facilitating analysis, data manipulation and data cleaning. Seaborn and Matplotlib were employed for creating visualizations.

The three datasets, along with tweets.csv, were imported using the code `pd.read_csv()` and `pd.read_excel()`, adhering to PEP8 guidelines by naming the DataFrames 'ad,' 'ar,' 'nc,' and 'tweets' corresponding to their file names. The exploration phase involved the application of functions such as `groupby()`, `sum()`, `nlargest()`, `nunique()`, `sort_values()`, `len()`, and `loc()` to unveil key insights into top locations, regions, service settings, and other relevant parameters.

Identification of trends emerged as a pivotal aspect of the analysis, utilizing the 'nc' and 'ad' DataFrames to delve into daily and monthly patterns. This incorporated grouping data and tallying the total number of appointments across various scenarios. It is noteworthy that Twitter analysis was intentionally set aside as it did not directly align with the primary focus of the study.

Python libraries Seaborn and Matplotlib facilitated data visualization. The decision to employ barplots and lineplots was grounded in their recognizability and simplicity for comparing various categories. Particularly, lineplots offered the advantage of comparing categories over a period. For the coding aspect of the visualizations, consistency in structure was maintained. After creating a graph, enhancements were made to the visual aspect, incorporating appropriate labels, removing grids, specifying a colour palette, customizing the legend, and adding data labels.

Insights

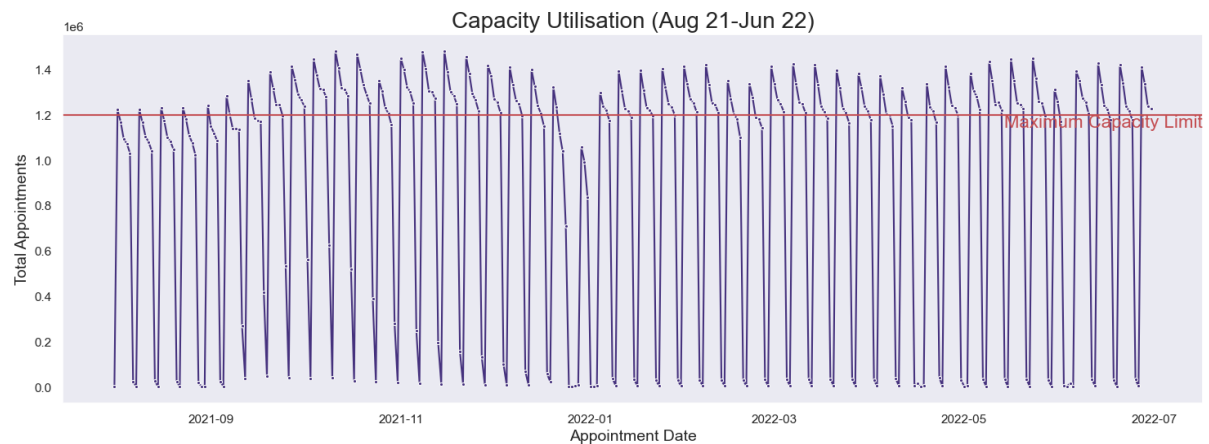
NHS Appointments

The onset of the first Covid-19 lockdown in March 2020 triggered a significant decline in NHS appointments, illustrating the dynamic impact of COVID-19 on scheduling trends. This fluctuating pattern correlated with the tightening and loosening of lockdown measures. As restrictions eased in July 2021, appointments surged, peaking in October, November, and March, with each month registering 30 million visits.



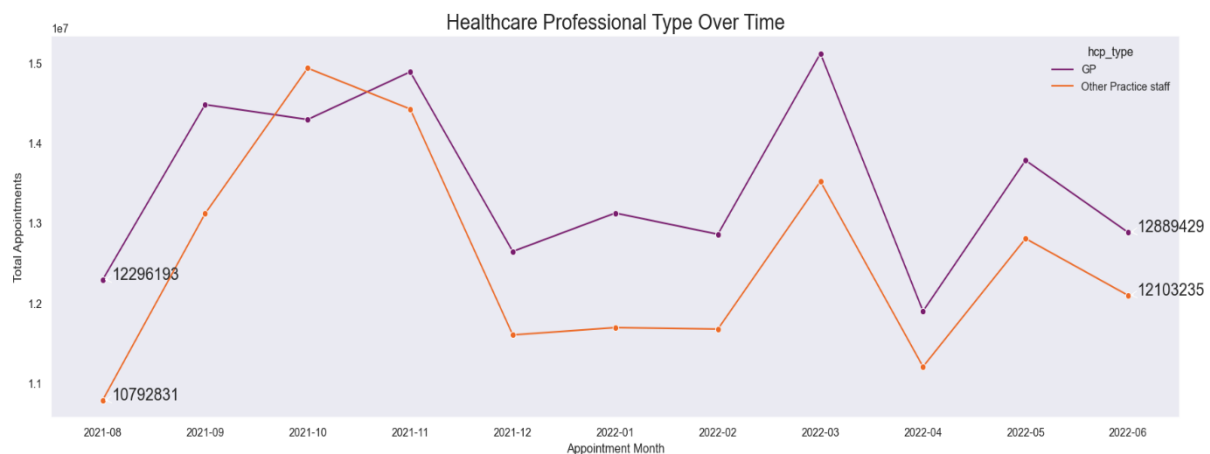
Capacity Analysis

The NHS, with a daily capacity limit of 1.2 million appointments, faced exceeding this capacity on 52% of the 334 days analysed. The early days of the week, particularly Tuesdays, emerged as the peak period, potentially influenced by bank holidays.



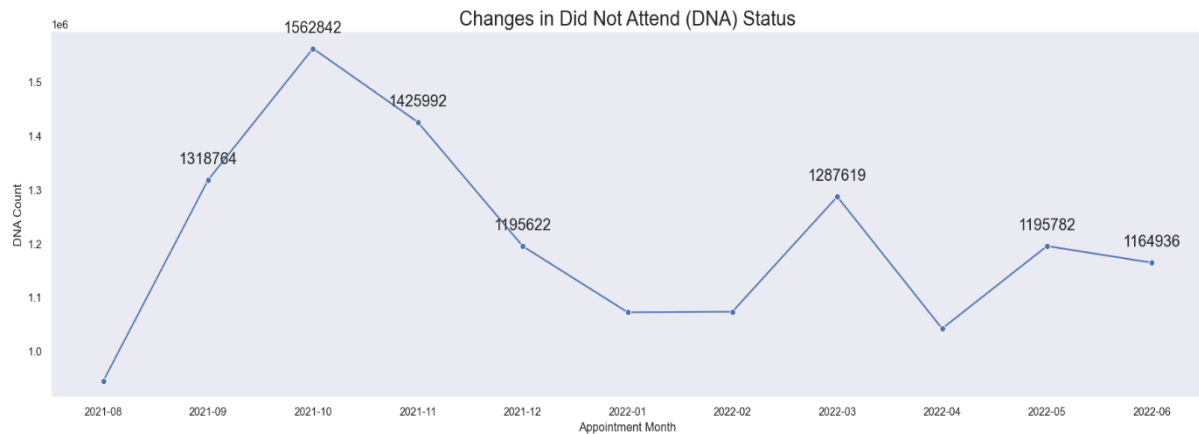
Healthcare Professional Types Analysis

General Practitioners (GPs) consistently dominated monthly appointment counts at 51.1%, while Other Practice Staff comprised 45.7%. Post-Covid restrictions removal, both GPs and other practice staff witnessed a surge in appointments. Notably, from August 2021, Other Practice Staff dealt with 1.3 million patients, surpassing the rise by 600,000 for GPs.



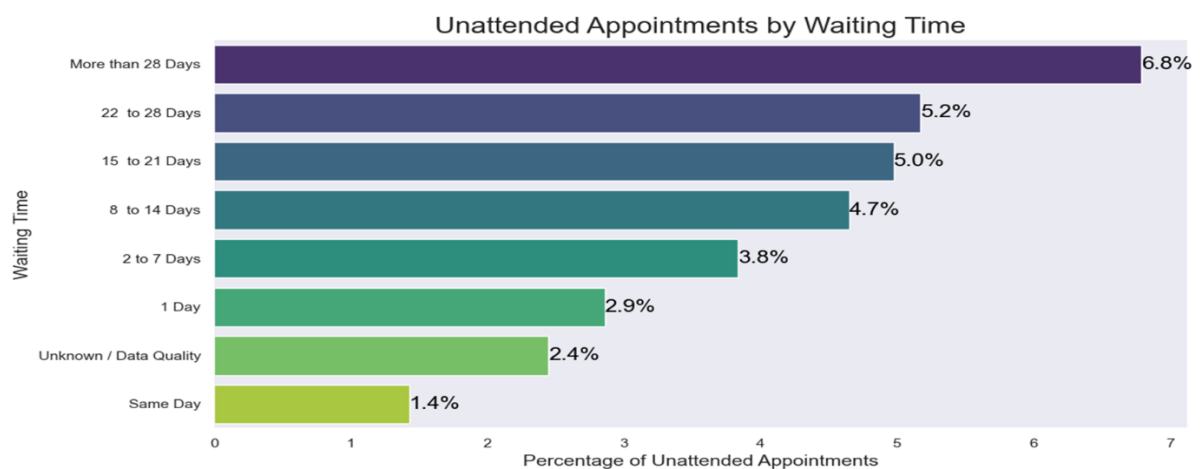
Attendance Analysis

While the majority (91.3%) of patients attended their appointments, a positive correlation was observed between non-attended appointments and the busiest months.



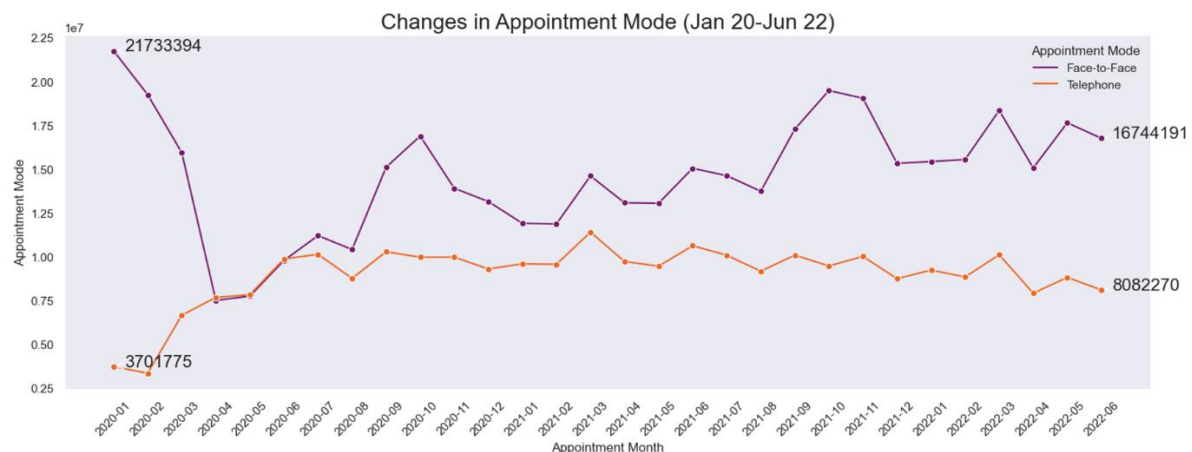
Trends in Time Between Booking an Appointment

While the majority (91.3%) attended their appointments, a positive correlation was observed between non-attended appointments and the busiest months (October, November, and March). An immediate preference for same-day appointments was evident in 44.3% of cases, while 20.5% fell within a 2 to 7 days waiting period, indicating a demand for relatively prompt consultations.



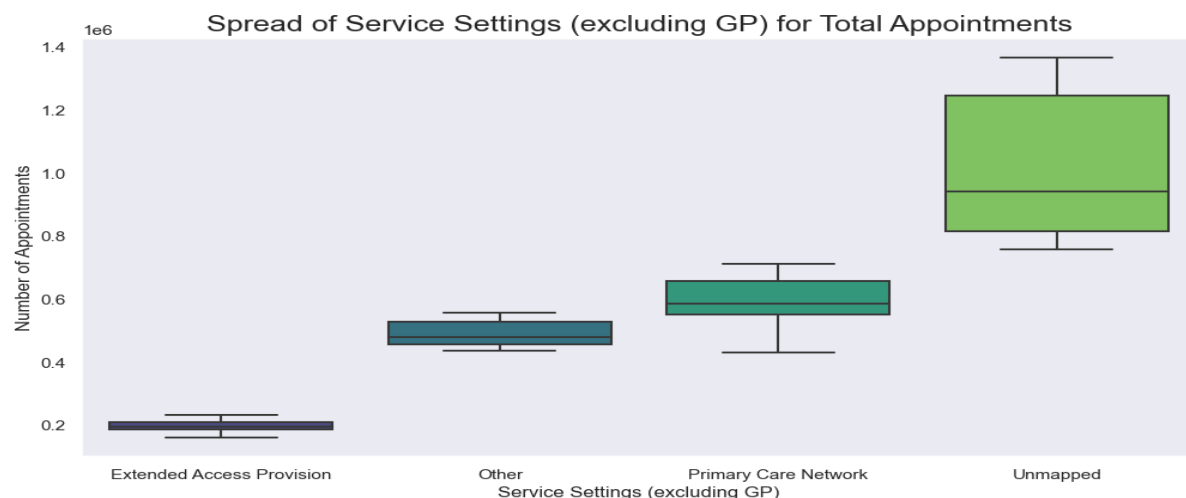
Trends in Appointment Type and the Busiest Months

Appointment modes distribution revealed "Face-to-Face" (59.2%) and "Telephone" (36.1%). Telephone appointments notably increased post-March 2020, while face-to-face appointments decreased by five million from January 2020 to June 2022. Despite a gradual shift back towards face-to-face appointments after Covid restrictions eased, they remain less common. The busiest months witnessed an increase in both telephone and face-to-face appointments, with face-to-face appointments experiencing a more significant rise.



Service Setting Analysis

NHS North East and North Cumbria emerged as the top ICB location, with the Midlands leading as the top region at 19.39%. General Practice consistently held the majority share of appointments at 91.5%, even post-Covid, with unmapped service settings exceeding one million appointments since August 2021, becoming the second-highest service category.



Recommendations

- **Increase Early Week Staffing:** Allocate more staff earlier in the week to manage the higher demand during the busiest months and extending staff availability on weekends to accommodate the demand for appointments.
- **Promote Telephone Appointments:** Given the consistent demand throughout the year and during busy times, encourage the use of telephone appointments where appropriate.
- **Same-Day Appointments:** Given that same-day appointments are attended the most, prioritize scheduling and promoting same-day appointments or keep the time between booking and the actual appointment as short as possible to improve attendance rates.
- **Patient Preferences:** Conduct consultations with patients to understand their preferences for appointment types and times to altering the scheduling system to better match patient needs and reduce missed appointments.
- **Online Scheduling:** Consider implementing an online scheduling system that allows patients to book appointments conveniently.

Reference

Office for National Statistics (Updated 2023). Health Geographies. Available at: <https://www.ons.gov.uk/methodology/geography/ukgeographies/healthgeography> (Accessed: 26 January 2024).

Guido van Rossum (2001). PEP 8 – Style Guide for Python Code. Available at: <https://peps.python.org/pep-0008/> (Accessed: 12 January 2024).

Pandas. Merge, join, concatenate, and compare. Available at: https://pandas.pydata.org/pandas-docs/stable/user_guide/merging.html (Accessed: 20 January 2024).

Visualization with Seaborn (VardePlas, 2021a). Available at: <https://jakevdp.github.io/PythonDataScienceHandbook/04.14-visualization-with-seaborn.html> (Accessed: 12 February 2024).

Appendix

Code for utilizing the top five sub ICB locations.	<pre># Determine the top five sub ICB locations based on appointments count. top_locations2 = nc.groupby('sub_icb_location_name')['count_of_appointments']/ .sum().nlargest(5) print('Top 5 sub ICB locations based on appointments count:') top_locations2</pre>
Code to integrate an external data source with the 'ar' dataset.	<pre># Import 'ICB_names.xlsx' as icb_names. icb_names = pd.read_excel('ICB_names.xlsx') # Merge ar_cleaned with ICB_names to add column for icb_names. ar_with_names = pd.merge(ar_cleaned, icb_names[['icb_ons_code', 'icb_names']],\ how='left', on='icb_ons_code') ar_with_names.info()</pre>
Example code for creating a bar plot.	<pre>import seaborn as sns # Set the plot style as darkgrid. sns.set_style('darkgrid') # Sort the dataframe by 'Total Appointments' in descending order and select the top 10. top_ten_nc_count = nc_count.sort_values('Total Appointments', ascending=False).head(10) # Create a horizontal bar chart using Seaborn. plt.figure(figsize=(15, 8)) ax = sns.barplot(x='Total Appointments', y='National Category', data=top_ten_nc_count, palette='viridis_r') # Remove grid lines. ax.grid(False) # Add percentage labels. for bar, percentage in zip(ax.patches, top_ten_nc_count['Percentage of Appointments']): plt.text(bar.get_width(), bar.get_y() + bar.get_height()/2, '{:.1f}%'.format(percentage), va='center', ha='left', color='black', fontsize=14) # Set Labels and title. plt.xlabel('Total Appointments') plt.ylabel('National Category') plt.title('Total Appointments by Top 10 National Categories', fontsize=14) # View and save the plot. plt.show() plt.tight_layout() plt.savefig('total_app_nc.png', dpi=96)</pre>
Code for grouping data and calculating percentages.	<pre># Group by 'hcp_type' and calculate the sum of 'count_of_appointments'. # The healthcare professional type named as 'hcp_type'. hcp_type = ar_cleaned.groupby('hcp_type')['count_of_appointments'].sum() \ .sort_values('count_of_appointments', ascending=False) \ .reset_index() # Calculate the total appointments count. total_hcp_type_count = ar_cleaned['count_of_appointments'].sum() # Calculate the percentage of total appointments for each healthcare professional type. hcp_type['percentage_of_appointments'] = (hcp_type['count_of_appointments'] / total_hcp_type_count) * 100 # Round the percentage values to one decimal place. hcp_type = hcp_type.round(1) # View the output with added percentage columns. hcp_type</pre>
Code example for tracking daily visits and utilization.	<pre># Plot sum of count of daily visits and utilisation. plt.figure(figsize=(12, 6)) sns.lineplot(x='appointment_month', y='utilisation', data=ar_df, marker='o', label='Utilisation') sns.barplot(x='appointment_month', y='count_of_appointments', data=ar_df, color='grey', label='Daily Visits') # Highlight days exceeding the limit high_utilisation_days = ar_df[ar_df['utilisation'] > 1200000] sns.scatterplot(x='appointment_month', y='utilisation', data=high_utilisation_days, color='red', s=50, label='High Utilisation') plt.title('Daily Visits and Utilisation with Focus on Days Exceeding 1,200,000 Appointments') plt.xlabel('Appointment Date') plt.ylabel('Count of Appointments / Utilisation') # Show the 1,200,000 appointments limit plt.axhline(y=1200000, color='r', linestyle='--', label='Limit') # Turn off grid lines and show the legend. plt.grid(False) plt.legend() plt.tight_layout() plt.savefig('daily_visits_utilisation_ar', dpi=96) plt.show()</pre>

Example code for creating a line plot.

```
# Create a line plot for changes in appointment mode over time.
plt.figure(figsize=(18,6))
sns.lineplot(x='appointment_month', y='count_of_appointments', hue='appointment_mode', \
             data=ar_mode_filt, marker='o', palette='inferno')

# Set labels and title.
plt.xlabel('Appointment Month')
plt.ylabel('Appointment Mode')
plt.title('Changes in Appointment Mode (Jan 20-Jun 22)')
plt.xticks(rotation=45)
plt.legend(title='Appointment Mode')

# Add data labels to the first and last points.
for mode in ar_mode_filt['appointment_mode'].unique():
    first_point = ar_mode_filt[ar_mode_filt['appointment_mode'] == mode].iloc[0]
    last_point = ar_mode_filt[ar_mode_filt['appointment_mode'] == mode].iloc[-1]

    plt.annotate(f'{first_point["count_of_appointments"]}',
                xy=(first_point['appointment_month'], first_point['count_of_appointments']),
                xytext=(10, 0), textcoords='offset points',
                arrowprops=dict(arrowstyle="->"))

    plt.annotate(f'{last_point["count_of_appointments"]}',
                xy=(last_point['appointment_month'], last_point['count_of_appointments']),
                xytext=(10, 0), textcoords='offset points',
                arrowprops=dict(arrowstyle="->"))

# Turn off grid lines.
plt.grid(False)

# Show and save the plot.
plt.show()
plt.savefig('ar_changes_in_apmode', dpi=96)
```

Example code for creating a box plot.

```
# Create a boxplot to investigate the service settings without GP.
# Filter out GP service setting.
nc_no_gp = nc_df[nc_df['service_setting'] != 'General Practice']

# Create a boxplot.
plt.figure(figsize=(12, 6))
sns.boxplot(x='service_setting', y='count_of_appointments', data=nc_no_gp, palette='viridis')

# Set labels and title
plt.xlabel('Service Settings (excluding GP)')
plt.ylabel('Number of Appointments')
plt.title('Spread of Service Settings (excluding GP) for Number of Appointments')
plt.grid(False)

# Show and save the plot
plt.show()
plt.savefig('spread_of_ss_noGP', dpi=96)
```